
ProjectName – C/Java/Python

Rapport préliminaire

Étudiant1, Étudiant2, Étudiant3, Étudiant4, Étudiant5

28 janvier 2026

Introduction

1 Algorithmes et Structures de Données

L'implémentation d'un joueur artificiel performant au Scrabble repose sur la résolution de deux problèmes distincts : la génération efficace de tous les coups légaux et la sélection du meilleur coup parmi ceux-ci. Cette section détaille les choix algorithmiques retenus pour répondre à ces besoins.

1.1 Modélisation du problème

Contrairement aux Échecs ou au Go, qui sont des jeux à information parfaite et déterministes, le Scrabble est un jeu à information imparfaite, on ne connaît pas lettres de l'adversaire et le tirage des lettres repose sur le hasard.

Une approche gloutonne, qui consisterait à jouer systématiquement le coup rapportant le plus de points à l'instant t , est sous-optimale. Elle néglige deux facteurs cruciaux :

- **Le reliquat (Rack Leave)** : La qualité des lettres conservées pour le tour suivant.
- **L'ouverture du plateau** : Le risque d'offrir une case "Mot Compte Triple" à l'adversaire.

Pour pallier ces manques, nous avons deux algorithmes de théorie des jeux : Minimax et Expectiminimax.

1.2 L'Algorithme Minimax

L'algorithme Minimax est la base de la décision dans les jeux à somme nulle. Il parcourt l'arbre de jeu jusqu'à une profondeur d donnée.

Fonctionnement

L'algorithme alterne entre deux types de nœuds :

- **Nœuds MAX** : Cherche à maximiser le score final.
- **Nœuds MIN** : Cherche à minimiser le score.

L'application stricte de Minimax au Scrabble pose un problème de modélisation. Minimax suppose que l'adversaire jouera toujours le coup optimal pour nous contrer. Or, pour déterminer ce coup optimal, l'algorithme doit connaître les lettres de l'adversaire. Dans une simulation Minimax, nous devrions supposons le pire cas : l'adversaire possède exactement les lettres nécessaires pour réfuter notre coup (ex : un "Z" pour atteindre un mot compte triple ouvert). Cette approche rend l'IA excessivement défensive et inadaptée à la réalité probabiliste du tirage.

1.3 L'Algorithme Expectiminimax

Pour intégrer la dimension aléatoire du tirage de lettres, nous utilisons l'algorithme **Expectiminimax**. C'est une variation du Minimax qui introduit un troisième type de nœud : les **Nœuds de Hasard**.

L'arbre de décision se décompose désormais en trois niveaux par tour de jeu :

1. **Nœud MAX** : L'IA choisit un coup parmi les coups possibles générés.
2. **Nœud CHANCE** : Ce nœud représente l'incertitude sur les lettres de l'adversaire. Il possède une branche pour chaque tirage possible, pondérée par sa probabilité.
3. **Nœud MIN** : L'adversaire joue le meilleur coup possible avec le tirage attribué par le nœud chance.

La valeur d'un nœud de hasard n'est ni un minimum ni un maximum, mais une espérance mathématique. Pour un état s correspondant à un nœud de hasard :

$$Expect(s) = \sum_{r \in Tirages} P(r) \times v(successeur(s, r))$$

Où $P(r)$ est la probabilité que le tirage soit r , sachant les lettres déjà visibles.

Défi combinatoire et Échantillonnage

Le facteur de branchement aux nœuds de hasard est gigantesque. Simuler tous les tirages possibles de 7 lettres parmi les N lettres restantes est impossible en temps réel.

Pour contourner cette complexité, nous utiliserons une méthode d'approximation par échantillonnage. Au lieu de calculer la somme sur tous les tirages, nous générons un sous-ensemble représentatif de N mains adverses aléatoires (ex : 20 simulations) basées sur la distribution réelle des lettres restantes.

L'algorithme devient alors :

1. L'IA joue un coup C .
2. Nous simulons N mains adverses possibles.
3. Pour chaque main, nous calculons la meilleure réponse de l'adversaire.
4. La valeur du coup C est la moyenne des scores résultant de ces simulations.

2 Spécifications étendues

F16 : Proposer de sauvegarder avant de quitter

Description : Le programme doit demander à l'utilisateur s'il souhaite sauvegarder la partie avant de quitter, si des modifications non sauvegardées existent.

Prérequis :

- F2 : Fichier de configuration (pour définir les chemins de sauvegarde par défaut).
- F21 : Format de sauvegarde (pour savoir comment structurer les fichiers de sauvegarde).
- F15 : Shell interactif (pour gérer les commandes comme quit et save).

Type de besoin :

Fonctionnel.

Découpage en sous-besoins :

Pour implémenter cette fonctionnalité, nous avons identifié quatre sous-besoins principaux.

Sous besoins 1. Détecter les modifications non sauvegardées

Description : Le programme doit détecter si l'état actuel du jeu diffère de l'état sauvegardé. Cela inclut les modifications du plateau, des scores et des lettres restantes.

Actions :

- Stocker l'état du jeu après chaque sauvegarde.
- Comparer l'état actuel avec l'état sauvegardé à chaque tentative de quitter le programme.

Fonction associée :

```
1 boolean modificationsNonSauvegardees(EtatJeu etatActuel, EtatJeu etatSauvegarde)
```

Paramètres :

- `etatActuel` : Représente l'état actuel du jeu (plateau, scores, lettres restantes).
- `etatSauvegarde` : Représente l'état du jeu lors de la dernière sauvegarde.

Retour :

- `true` : S'il y a des modifications non sauvegardées.
- `false` : Si aucun changement n'a été fait depuis la dernière sauvegarde.

Test unitaire :

- **Cas 1** : `etatActuel` contient le mot "CHAT" sur le plateau, `etatSauvegarde` est vide. Résultat attendu : `true` (car il y a des modifications).
- **Cas 2** : `etatActuel` et `etatSauvegarde` sont identiques. Résultat attendu : `false` (pas de modifications).

Sous besoins 2. Demander confirmation à l'utilisateur

Description : Le programme doit afficher un message de confirmation et lire la réponse de l'utilisateur.

Actions :

- Afficher le message : `Save the game before quitting? [y/N].`
- Lire la réponse de l'utilisateur.
- Retourner `true` si la réponse est "y" ou "Y", `false` sinon.

Fonction associée :

```
1 boolean demanderConfirmation()
```

Retour :

- `true` : Si l'utilisateur répond "y" ou "Y".
- `false` : Si l'utilisateur répond "n", "N" ou toute autre réponse.

Test unitaire :

- **Cas 1** : L'utilisateur répond "y". Résultat attendu : `true`.
- **Cas 2** : L'utilisateur répond "n". Résultat attendu : `false`.

Sous besoins 3. Sauvegarder la partie

Description : Si l'utilisateur choisit de sauvegarder, le programme doit écrire l'état actuel du jeu dans un fichier.

Actions :

- Demander à l'utilisateur le chemin du fichier de sauvegarde.
- Écrire l'état du jeu dans le fichier au format défini (JSON ou format personnalisé).

Fonction associée :

```
1 boolean sauvegarderPartie(EtatJeu etatJeu, String cheminFichier)
```

Paramètres :

- `etatJeu` : L'état actuel du jeu à sauvegarder.
- `cheminFichier` : Le chemin du fichier où sauvegarder la partie.

Retour :

- `true` : Si la sauvegarde a réussi.
- `false` : Si une erreur survient.

Test unitaire :

- **Cas 1** : Sauvegarder un état de jeu valide dans un fichier accessible. Résultat attendu : `true` et le fichier contient les données correctes.
- **Cas 2** : Sauvegarder dans un chemin invalide (ex. : `/dossier/inexistant/fichier.json`). Résultat attendu : `false`.

Sous besoins 4. Gérer les erreurs de sauvegarde

Description : Le programme doit gérer les erreurs lors de la sauvegarde (ex. : permissions insuffisantes, disque plein) et redemander à l'utilisateur s'il souhaite réessayer.

Actions :

- Capturer les exceptions lors de l'écriture du fichier.
- Afficher un message d'erreur et redemander à l'utilisateur s'il veut réessayer.

Fonction associée :

```
1 boolean gererErreurSauvegarde(EtatJeu etatJeu)
```

Paramètres :

- `etatJeu` : L'état actuel du jeu à sauvegarder.

Retour :

- `true` : Si la sauvegarde a finalement réussi.
- `false` : Si l'utilisateur abandonne après une erreur.

Test unitaire :

- **Cas 1** : L'utilisateur entre un chemin invalide, puis un chemin valide. Résultat attendu : `true` après la deuxième tentative.
- **Cas 2** : L'utilisateur abandonne après une erreur. Résultat attendu : `false`.